

1주차 강의

OS란?

Operation system : application을 돕는 프로그램(비공식적 정의)

어떻게 CPU, 메모리, 저장장치를 사용할까?

어떻게 사용자 입력을 받을 것인가?(키보드, 마우스, 터치스크린)

어떻게 사용자에게 결과를 알려줄 것인가?(디스플레이, 소리, 햅틱)

Various OS : Windows 11, Mac OS, Linux

OS는 어디에나 있다. 컴퓨터, 스마트 TV, 스마트폰, 자동차, 스마트홈

운영체제의 역할

컴퓨터 자원을 관리한다. e.g. 메모리, CPU, 저장장치

컴퓨터에서 SW와 HW를 연결함

컴퓨터 내부 부품과 외부 장치들이 서로 통신하도록 해 줌

OS 보안

메모리 보호, 디스크 보호, 접근 제어, 사용자 인증

CrowdStrike 장애

2024. 7. 19. 보안 업데이트 이후, 윈도우 PC가 무한 재부팅 문제를 일으켰고,

850만 대의 컴퓨터에 영향을 미침.

은행, 항공사, 방송국, 슈퍼마켓 등 많은 곳이 영향을 받음.

블루스크린이 떴서 대규모 피해를 입혔고, 안전모드로 부팅한 후에 문제되는 파일 제거를 통해서 문제를 해결함.

OS를 배워야 하는 이유

컴퓨터에서 제일 중요한 프로그램

코딩을 넘어서 문제 해결과 관련. -> OS나 HW 문제도 잘 알아야 함.

보안 문제는 OS와 매우 밀접함.

Concurrency 문제 - 여러 작업이 동시에 실행되면서 충돌하는 문제. 자원 동시 접근, deadlock 등...

Stack overflow 문제

디지털 포렌식

정보 유출

인증, 권한부여, 가용성 문제 Authentication, Authorization, Availability 문제

클라우드 시스템도 운영체제와 매우 밀접함.

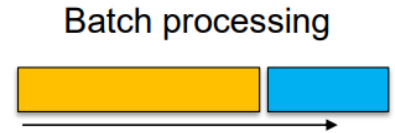
가상화

2주차 강의

컴퓨터의 역사

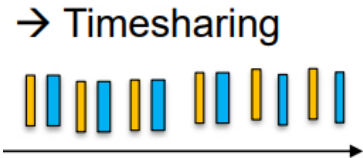
1960s :

Main Frame Computer 메인 프레임 컴퓨터 크기 방만함
IBM system/360 - 아직 금융권에서 쓰이고 있음. **안정성이 높음.**
Batch Program : 줄 세워놓고 차례로 처리



1970s :

Mini Computer 미니 컴퓨터 옷장 크기정도
PDP-11
배치 처리 -> **타임쉐어링(timesharing)**으로 발전



: 작업이 끝날 때까지 기다리는 것이 아니라, 여러 작업에 CPU 시간을 조금씩 번갈아 주는 방식
사용자는 멈춘 것처럼 느끼지 않고, interactive한 느낌을 받게 됨

멀티 프로그래밍 가능

Unix OS 등장함.

CLI 환경 사용.

1980s :

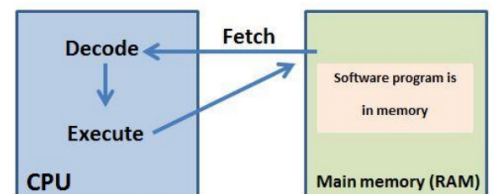
개인용 컴퓨터(PC) : Dos, Windows, Mac OS, Linux

CLI -> GUI Mac이 처음 GUI 출시 -> 윈도우가 확산시킴

Linux는 무료 오픈소스 OS -> 보안, 서버, 클라우드, 모바일 분야로 갈수록 Linux를 다룰 줄 아는 것이 중요
기업용 **redhat** 사용자 친화적 **Ubuntu** 보안 특화 **Kali Linux**

프로그램이 실행될 때 일어나는 일

1. **Fetch** : CPU가 메모리에서 명령어를 가져옴
2. **Decode** : 그 명령어가 무엇인지 해석함
3. **Execute** : 실제로 수행함 e.g. 두수 더하기, 메모리 접근, 조건 검사, 분기 처리, 함수 호출, 점프
4. **Next instruction** : 다음 명령어로 넘어감
5. 이 과정을 계속 반복



OS의 역할

프로그램 실행을 쉽게 해 줌

여러 프로그램이 메모리를 사용할 수 있게 관리함

장치와 상호작용할 수 있게 해 줌

-> 시스템이 정확하고(correctly) 효율적으로(efficiently) 동작하게 함

OS가 다루는 자원

CPU

프로그램을 실행하는 주체. 코드와 데이터를 바탕으로 실제 연산수행

메모리

코드와 데이터를 읽고 씀. 실행 중인 프로그램이 당장 사용하는 공간

스토리지

영구적으로 코드와 데이터 보관. 전원이 꺼져도 남아 있어야 하는 정보 저장.

컴퓨터 처음 켤때도 가져와야 함.

Q) CPU와 스토리지만 있으면 되지 왜 메모리가 필요할까?

-> 스토리지는 메모리보다 훨씬 느리기 때문. 반드시 메모리가 필요. 실행속도 확보 차원.

CPU안의 아주 작은 메모리(레지스터)도 있고, 캐시(레지스터와 메모리 사이)

가상화 Virtualization

운영체제가 물리 자원(physical resource)을 프로그램이 쓰기 쉬운 가상 형태(virtual form)로 바꾸어 제공하는 것

물리 자원 : CPU/프로세서, 메모리, 디스크 등..

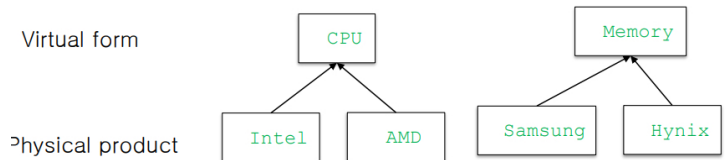
가상화를 하는 것이 더 General하고, Powerful하고 사용하기 더 쉬움

운영체제를 Virtual Machine이라고 볼 수도 있음.

가상화를 하면 독점적 사용을 하는 것과 같은 illusion(환상)을 준다.

->실제로는 여러 프로그램이 자원을 쓰고 있지만, 각 프로그램은 자기가 혼자 쓰는 것처럼 느끼게 만든다.

어떤 프로그램을 만들 때 다른 프로그램의 존재까지 다 의식하면 코드가 너무 복잡해짐. 따라서 앱을 만들 때에는 "이 컴퓨터를 독점적으로 사용한다고 가정하는 편이 훨씬 쉬움.



OS의 설계 목표

1. Abstraction 추상화

시스템을 편리하고 쉽게 사용하게 함.

e.g. 사용자가 복잡한 행정적 절차를 직접 다 처리하는 것이 아니라, 학과 조교나 행정 담당에게 요청하면 뒤에서 알아서 처리해 주는 것과 비슷.

OS는 자주쓰는 기능을 묶어둔 라이브러리같은 것에서 출발해서 점점 더 큰 시스템으로 발전해 왔음.

2. High Performance

OS의 오버헤드를 최소화 해야함. OS는 가상화를 제공하되 지나치게 느려지면 안됨.

3. Protection between Application 앱 간 보호(격리)

한 앱의 잘못된 동작이 다른 앱이나 OS 자체를 망가뜨리면 안 됨

Isolation : 여러 앱이 동시에 실행되더라도 서로 침범하지 않게 보호해야 함

4. Reliability 신뢰성

운영체제는 가능한 한 멈추지 않고 계속 동작해야 함

자동차, 임베디드, 사이버 물리 시스템에서 OS가 멈추는 것은 훨씬 더 위험함 -> 안전과 직결

5. 기타

에너지 효율, 보안, 이동성(mobility).

스마트폰 배터리가 오래 가나?

Three Pieces of Operating Systems

OS 3요소

1. Virtualization : 물리 자원을 독점적으로 쓰는 것처럼 보이게 하는 것 exclusive use의 illusion

2. Concurrency : 여러 작업이 동시에 진행되는 문제. multi-tasking과 연결됨

3. Persistence : 계산 결과를 영구 저장장치에 기록하는 문제. 파일, 디렉터리, I/O와 연결됨

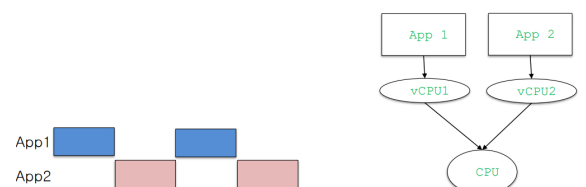
CPU 가상화

시스템은 다수의 가상 CPU들을 가지고 있다.

OS가 앱마다 vCPU(virtual CPU)가 있는 것처럼 보이게 만들.

실제로는 하나의 CPU를 잘게 나눠 번갈아 사용하게 함

앱이 10개면 OS가 vCPU 10개 준 것처럼 보임



3주차 강의

CPU의 가상화

```
#include <stdio.h> #include <stdlib.h> #include <sys/time.h> #include <assert.h> #include "common.h"
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\\n");          //argument가 2개가 아니면 에러 출력!
        exit(1);
    }
    char *str = argv[1];
    while (1) {
        Spin(1);          // 1초동안 일을 함
        printf("%s\\n", str);    // 사용자가 입력한 str 출력
    }
    return 0;
}
```

예시 입출력 1

```
prompt> gcc -o cpu cpu.c -Wall          // cpu.c 파일을 컴파일해서 cpu 실행파일 만들기
prompt> ./cpu "A"                      //인자 "A"를 주고 cpu 파일 실행
A                                       //
A
A
^C                                     // Ctrl + C 강제종료
prompt>
```

```
./cpu A &   ./cpu B &   ./cpu C &   ./cpu D &          //&을 붙이면 백그라운드에서 실행!
[1] 7353     //터미널에서 다른 명령 입력 가능
[2] 7354
[3] 7355
[4] 7356
A B D C A B D C A B D C ...          //누가 먼저 CPU를 받느냐에 따라 출력 순서가 조금씩 달라질 수 있다
```

프로세서가 하나뿐이어도, 네 개의 프로그램이 동시에 실행되는 것처럼 보인다!

ps 명령으로 확인해 보면 여러개가 동시에 돌아가는 것처럼 보임. CPU 시간이 비교적 공평하게 분배.

CPU 공유하는 방법 2가지

1. **앱끼리 직접 조정** : 내가 먼저 쓰고 너 쓰고, 다시 내가 쓰는 식으로 앱들이 직접 규칙 정하는 방식
 2. **OS가 조정** : 앱은 자기가 CPU를 다 가진 것처럼 행동, OS가 알아서 중재.
- > 두 번째가 더 나은 방식이다!

KTX 좌석표 비유 : 같은 좌석 번호를 가진 표 두 장이 생겼다고 하면, 승객끼리 직접 표를 비교하기 보다는 **승무원**을 부르면 간단하다! **승무원(OS)**가 무슨 문제인지 파악하고 중재를 해 준다.

메모리 가상화

physical memory는 바이트(byte)의 배열이다.

- load(read) : 주소를 주고 그 위치의 내용을 읽음
- store(write) : 특정 주소에 데이터를 씀

program : 디스크에 저장된 코드, **Passive**한 존재

process : **실행 중인 프로그램**, **Active**한 존재

각 프로세스는 자기만의 **virtual memory address space**를 가진다.

OS가 이 가상 주소 공간을 **physical memory**에 매핑한다.

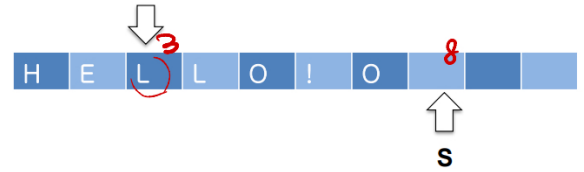
한 프로그램의 메모리 참조가 **다른 프로그램의 주소 공간에 영향**을 주지 않는다.

Physical memory는 공유되는 자원이고, OS에 의해서 제어된다.

앱이 운영체제 메모리 영역까지 침범하면 단순히 앱 하나가 죽는 게 아니라 시스템 전체가 망가질 수 있다.

메모리 isolation = 보안과 안정성의 핵심

1byte == 8 bits 각 비트는 0 또는 1
8비트이면 2의 8승, 256개의 경우를 표현 可



Concurrency 문제

프로세스는 여러 일을 한번에 다룸.

한 프로세스를 실행하다가 다른 일을 하고, 다시 다른 프로세스로 넘어가는 식으로 juggling을 함.

현대 multi-threaded programs도 동시성 문제를 가지고 있다.

현대의 프로그램은 단일 흐름만 가지지 않는다.

여러 개의 실행 흐름이 동시에 진행되는 multi-threaded 구조가 많다.

크롬 브라우저 : 멀티스레드가 아니었다면 한 창이 완전히 뜯 뒤 다음 창이 뜨는 식으로 훨씬 답답했을 것이다.

싱글 코어 CPU : 한 번에 하나의 작업만 직접 처리한다. -> 이 수업에서는 싱글코어 관점에 집중

멀티 코어 CPU : 코어 수만큼 동시에 일을 처리할 수 있다. Intel i9에는 코어가 24개가 있다.

GPU : 병렬 처리 특화. 각 코어 하나하나의 성능은 CPU보다 약하지만, 개수가 매우 많음(매우 많은 초등학생)

RTX 5090에는 코어가 21,760개 있음

Persistence 영속성

DRAM 같은 장치는 **volatile**하다.

HW와 SW는 지속적으로 데이터를 저장할 저장 공간이 필요하다.

HW : HDD, SSD 같은 I/O 디바이스

SW : 파일 시스템이 disk를 관리한다. 파일 시스템은 사용자가 만든 어떤 파일이든 저장할 책임이 있다.

OS가 disk에 쓰기 위해서 하는 일은?

새로운 데이터를 어디에 쓸지 결정.

I/O 요청을 저장장치에 보냄

파일 시스템이 쓰는 도중에 **crash**나는 것을 **handling**한다

- Journaling or Copy-on-write을 한다.

Journaling : 원래 위치를 바로 덮어쓰지 않고, 뒤에 “이 내용을 여기에 써야 한다”는 식으로 journal 이라는 공간에 추가 기록을 남기는 방식. 히스토리를 관리하는 느낌.

Copy-on-write: 기존 데이터를 바로 덮어쓰지 말고, 새 위치에 복사본을 써 놓은 뒤, **마지막에 연결만 바꾸자**

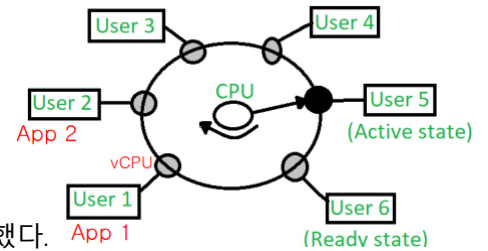
복숭아 이야기

주제 : 가상화(virtualization)는 실제 자원은 적은데, 각 프로그램이나 사용자는 마치 자기만의 자원을 가진 것처럼 느끼게 만드는 기술이다.

복숭아 1개 = physical CPU 1개

사람마다 자기 복숭아가 있는 것처럼 느끼는 상태 = 각 프로그램이 자기 CPU가 있는 것처럼 느끼는 상태

실제 독점이 아니라 독점이라고 느끼는 illusion



영화 "Her"

주인공은 Samantha가 자기에게만 존재한다고 느낀다.

Samantha는 8,316명과 대화하고 그중 641명을 사랑한다고 나온다고 설명했다.

-> 사용자는 "나만 쓰고 있다"고 느끼지만, 실제 시스템은 다수 사용자에게 동시에 서비스를 제공할 수 있다

4주차 강의

Virtualization의 역사

1960s : Batch System

메인프레임 컴퓨터 1대를 많은 사용자가 공유.

입력도 순차적, 출력도 순차적. 천공 카드, 테이프 등을 이용해 작업을 넣고 결과를 기다리는 구조
무한정 대기. 한 곳에서 딜레이가 생기면 계속 딜레이

1970s : Time Sharing

여전히 machine은 적고 user는 많지만 CPU 작업별로 조금씩 나누어 사용하게 됨.
각 사용자는 터미널을 통해 원격으로 접속하고, 중앙의 머신이 CPU 시간을 배분함
"내가 지금 이 컴퓨터를 쓰고 있다"고 느끼게 됨"

1990s : Desktop Era

Many App – 1 CPU

"사용자 수"가 아니라 "앱 수"가 많아짐

한 사용자의 컴퓨터 안에서도 CPU 공유 문제가 다시 나타남.

2000s : Cloud Era

Many Server – 1 HW

하나의 HW 위에 여러 서버를 가상화하여 동작시킴

Hypervisor 위에 여러 VM을 두고, 어떤 VM은 Windows, 어떤 VM은 Linux를 돌리는 형태가 가능함

저성능 서버 여러 대를 따로 관리하는 것보다,

고성능 서버 하나를 두고 가상화해서 관리하는 편이 훨씬 편리하기 때문

클라우드는 자원을 유연하게 늘릴 수 있어서 선호

클라우드가 한 번 장애가 나면 파급력이 너무 커서 뉴스에 크게 나온다. -> 한번 문제 생기면 파급력 廣

CPU 가상화

운영체제는 많은 virtual CPU가 있는 것처럼 보이게 함

수업에서 말한 "1초"는 실제 정확한 1초라기보다, 이해를 돕기 위한 one-time unit 개념

Process

실제로 메모리에 올라와 CPU에서 실행되고 있는 프로그램

- Memory(address space)

- * instructions
- * data section

- Registers

- * **program counter(PC)** : 다음 instruction의 위치
- * **stack pointer(SP)** : 현재 stack의 위치를 가리킴. 현재 스택의 top을 가리킴.

지금 어느 함수가 실행중인지, 함수 parameter, local variable, return address가 담겨 있음.

Process States

Running : 프로세스가 프로세서에서 돌아가고 있는 것

Ready : 준비는 되었지만, 무슨 이유에선지 OS가 CPU를 배정해 주지 않은 상태

Blocked(Waiting) : I/O 요청 등 때문에 기다리는 상태.

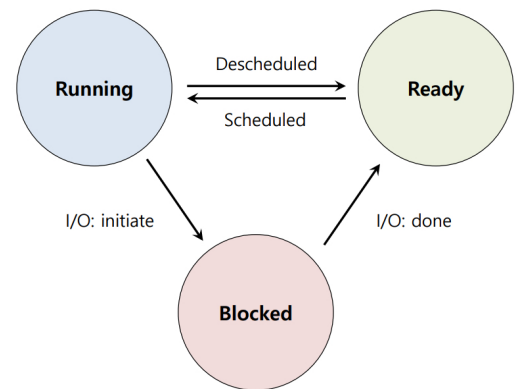
Process State Transition

Scheduled : OS가 이 프로세스를 선택하면 running이 됨

Deschedule : OS가 공평성을 위해 CPU를 강제로 빼앗음 **timeout**.

I/O initiate : I/O를 시작하면 그 작업이 끝날 때까지 Blocked!

I/O done : I/O가 끝나면 다시 실행할 준비가 되었으므로 ready.



* block에서 running으로 갈 수는 없다.

★실제 컴퓨터에서는 CPU 작업보다 I/O작업이 훨씬 많다. 대부분의 작업이 I/O 중심
I/O의 종류 : disk 읽기/쓰기, 키보드 등 input, 모니터 등 output, 프로그램 로딩..

Process API

API : 프로그래밍을 할 때 어떻게 사용해야 하는가를 바깥으로 알려주는 것.

여기서는 process를 다루기 위한 공통적 인터페이스라는 뜻

아래 API는 현대 OS가 작동하게 함.

- **Create** : 새 프로세스 생성
- **Destroy** : runaway process 같은 것을 종료(halt)
- **Wait** : 어떤 프로세스가 끝날 때까지 기다림

Process Creation 프로세스 생성

1. 프로그램 코드를 메모리, 즉 프로세스의 주소 공간(address space)에 load

보통 프로그램은 disk에 실행파일 형태(executable format)으로 저장되어 있음.

운영체제는 적재(load) 과정을 **게으르게(lazily)** 수행한다

-> 처음부터 전부 다 올리지 않고, 필요할 때만 조금씩 올린다

2. 프로그램은 실행 시간 스택(runtime stack)이 할당된다.

-> 프로그램이 실행될 때 OS는 프로세스를 위한 스택 공간을 준비 해 놓는다.

지역변수, 매개변수, 복귀 주소 등을 저장해 놓음.

main() 함수의 인자 argc와 argv 배열로 스택을 초기화한다

OS가 main()에 전달할 인자를 준비해 놓음

프로세스 생성 Stack vs Heap

Stack

function 함수에 관한 정보를 저장
함수 매개변수, 지역변수, 리턴 주소

함수의 호출/리턴에 따라서 늘어나거나 줄어듦

Heap

명시적으로 요청한 동적 할당 데이터(예: 리스트, 버퍼) 저장
malloc()으로 호출해 공간을 요청하고, free()를 호출해 해제.

메모리 할당/해제에 따라 늘어나거나 줄어듦

1. 앱을 메모리로 load해 옴.

2. stack을 할당함

3. 프로그램이 heap이 생성됨

명시적으로 요청된 동적 할당 데이터를 위해서 사용됨.

4. OS는 다른 초기화 작업도 함

I/O 초기화 : 각각의 프로세스가 시작할 때 기본적으로 3개의 file descriptor를 가짐

file descriptor : OS가 프로세스에게 주는 입출력 통로 번호표

0 - standard Input

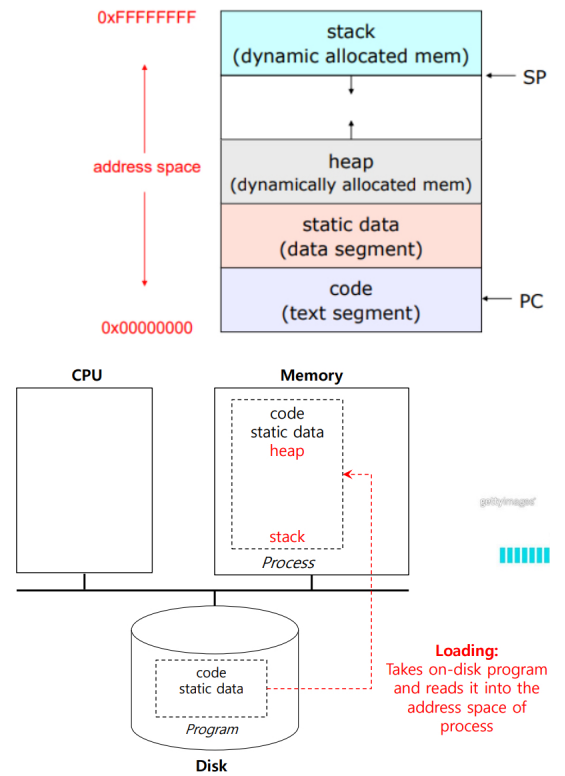
1 - standard Output

2 - standard Error

-> 리다이렉트(프로그램의 입력이나 출력을 원래 가던 곳이 아니라 다른 곳으로 돌리는 것)를 할 수 있다.

5. entry point인 main()부터 실행 시작.

OS는 CPU의 제어권을 새로 생성된 프로세스에게 넘긴다.



5주차 강의

시스템 콜

OS에 서비스를 요청하는 방법

ex) 파일 열기, 읽기, 쓰기, 메모리 할당 등...

프로세스는 어떻게 생성할까?

프로세스 생성은 User level에서 하는 것이 아님.

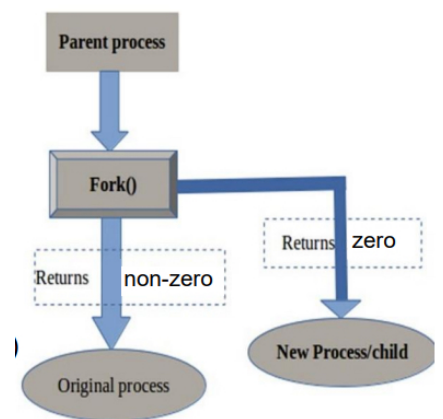
-> 프로세스를 생성하기 위해 시스템 콜을 함!

fork() 시스템 콜

현재 프로세스(parent)를 복사해서 새로운 프로세스(child process)를 만든다

fork() 이후 child와 parent는 독립적으로 실행된다

child와 parent가 서로 다른 동작을 하도록 fork()는 서로 다른 반환값을 준다



Child 프로세스의 생성

child는 parent와 별도의 메모리 공간을 할당받는다. 하지만 생성 직후 메모리 내용은 parent와 같다.

child process는 자기 자신의 레지스터들과 PC를 가진다

새로 만들어진 프로세스는 생성된 뒤 독립적이 된다

parent에게는 fork()가 child process의 PID를 반환하고, child에게는 fork()가 0을 반환한다

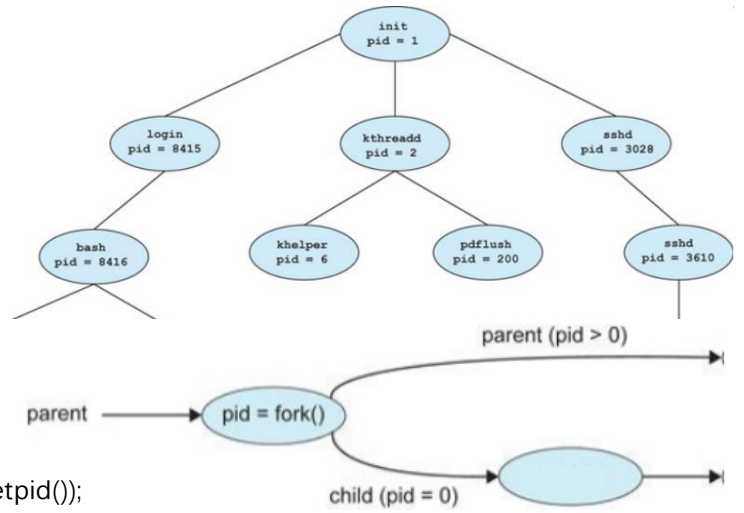
리눅스 커널 fork tree

init pid = 1

PID : 프로세스 ID

PPID : 부모 프로세스의 PID

```
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {                    // rc > 0, parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int) getpid());
    }
    return 0;
}
```



```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

or

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

출력은 어떻게 될지 모른다! 둘 중 하나 ----->

wait()

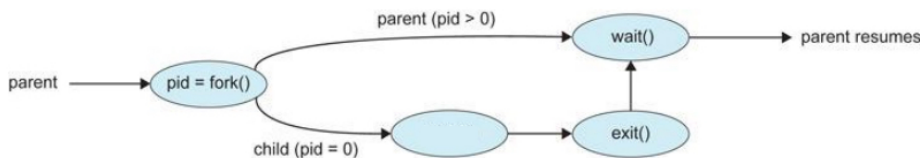
프로세스들 사이에 의존성(dependency)을 만든다

child 프로세스가 만들어졌을 때, **parent** **안의 wait()**는 child가 실행을 마치고 종료할 때까지 반환되지 않음

fork()만 하고 wait()을 하지 않으면, 원래 parent와 child는 아무 의존성이 없음.

어떤 경우에는 애플리케이션이 실행 순서를 강제하고 싶어 한다.

예를 들어 **parent**는 **child**가 끝난 뒤에만 종료하게 만들고 싶어 수 있다



int wc = wait(NULL); //wc에는 보통 종료한 child의 PID가 들어감.

```
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {                // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {                            // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n", rc, wc, (int) getpid());
    }
    return 0;
}
```

```
prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (wc:29267) (pid:29266)
prompt>
```

출력은 이렇게 강제됨!

exec()

: 호출자(caller)는 자기 자신과는 다른 프로그램을 실행하고 싶어 한다

e.g. Launch an editor 또는 ls -l

OS는 새 프로그램을 위해 새로운 binary image를 로드하고, 새로운 stack과 heap을 초기화해야 한다
현재 프로세스 안에 들어 있는 프로그램 내용을 새 프로그램 기준으로 통째로 바꾸는 것!

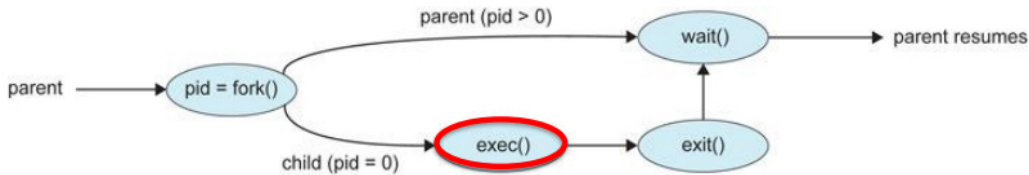
exec()는 돌아오지 않는다. 그 시점을 기준으로 완전히 덮어써버리기 때문!

apt-get

① 실행할 바이너리 파일의 이름

install python

② 인자(arguments) 배열



```
char *argv[3];
argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);      // /bin/echo라는 경로로 argv 인자를 넘겨서 실행하라
printf("exec error\n");      // exec()이 되면 이건 출력되지 않는다!
```

```
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());

    int rc = fork();
    if (rc < 0) { // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc"); // program: "wc" (word count)
        myargs[1] = strdup("p3.c"); // argument: file to count
        myargs[2] = NULL; // marks end of array
        execvp(myargs[0], myargs); // runs word count
        printf("this shouldn't print out");
    } else { // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
29 107 1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

fork()와 exec()

왜 그냥 `forkandexec("ls", "ls -l")` 같은 걸 쓰지 않는가? 왜 두개가 분리되어 있을까?

`fork()`와 `exec()`를 분리해 두면, 새 프로그램을 실행하기 직전에 여러 설정을 조작할 수 있고, 그 결과 I/O 리다이렉션과 파이프가 가능해진다

fork()와 exec() 사이의 짧은 구간이 중요하다!

이 구간에서 `child`는 아직 원래 프로그램 코드 안에 있으면서도, 곧 `exec()`로 새 프로그램으로 바뀔 준비를 하고 있어.

분리해 두면 새 프로그램을 실행하기 전에 `child` 쪽에서 여러 설정을 먼저 바꿀 수 있기 때문이라고 했고, 이 짧은 구간 덕분에 `child`의 환경을 원하는 대로 바꿔 놓고 `exec`할 수 있다.

1. I/O redirection

표준 IO가 아니라 새로운 IO 장치로 출력하라는 의미

```
cat w3.c > newfile.txt
```

`cat w3.c`는 원래 화면으로 출력이 되어야 하는데 `>` 을 통해서 redirect를 함!

2. pipe

한 프로그램의 출력(output)을 다른 프로그램의 입력(input)으로 연결

```
echo hello world | wc
```

`hello world`를 화면에 `echo` 하지 말고 그 출력을 `wc`의 입력으로 사용

6주차 강의

CPU를 효율적으로 virtualize하기 위해서는 어떻게 해야 할까?

OS는 물리 CPU를 time sharing을 한다.

성능 performance : 오버헤드를 더하지 않고 어떻게 가상화 성능을 높일 수 있을까?

제어 control : CPU control을 유지하면서 프로세스를 효율적으로 실행하는 방법은 뭘까?

Direct Exection

프로그램을 그냥 CPU에서 바로 실행시는 방법

OS	Program
1. Create entry for process list 2. Allocate memory for program 3. Load program into memory 4. Set up stack with <code>argc / argv</code> 5. Clear registers 6. Execute <code>call main()</code>	7. Run <code>main()</code> 8. Execute <code>return from main()</code>
9. Free memory of process 10. Remove from process list	

실행 중인 프로그램에 아무 제한이 없으면, OS는 아무것도 통제할 수 없고 결국 '그냥 라이브러리'가 된다.

Direct Execution의 문제점

1. User가 이상한 짓을 할 수 있다.
e.g. 건드리면 안 되는 메모리를 건드릴 수 있음
-> 시스템 전체를 망가뜨릴 수 있는 문제
2. CPU가 control을 뺏어오기가 어렵다.
e.g. 무한반복에 빠질 수 있음.

```
int *i ;  
i = 0 ;  
*i = 1 ;  
  
i = -1 ;  
while (i < 0)  
    do something;
```

문제 1 : 제한된 작업 Restricted Operation

만약 프로세스가 제한된 작업을 하고 싶으면 어떻게 해야 할까?

e.g. 디스크 I/O 요청

e.g. CPU나 메모리 같은 시스템 자원 추가 사용

방법 1 : 다 허용

방법 2 : 보호된 방식으로 제어를 넘겨줌

user mode 유저 모드 : 애플리케이션은 하드웨어 자원에 대한 완전한 접근 권한이 없다

kernel mode 커널 모드 : OS가 시스템 자원 전체에 접근할 수 있음.

시스템 콜 System Call

user program이 OS 기능을 사용하기 위해 보내는 공식 요청

- 파일 시스템 접근
- 프로세스 생성 / 삭제
- 다른 프로세스와 통신
- 메모리 추가 할당

시스템 콜의 개발

ATLAS : 가상 메모리(페이징), extracode(시스템 콜)을 가진 세계 최초의 슈퍼컴퓨터. in 영국
system call은 OS가 특권 작업을 안전하게 제공하기 위한 구조이다.

Trap instruction : 현재 실행 중인 사용자 프로그램을 멈추고 커널(OS)로 제어를 넘기게 만드는 것
특권 레벨 privilege level을 **kernel** 모드로 올린다.

kernel stack에 레지스터를 저장한다. (eip, cs, eflags, esp, ss)

kernel안에 목적지를 찾음

목적지로 jump함

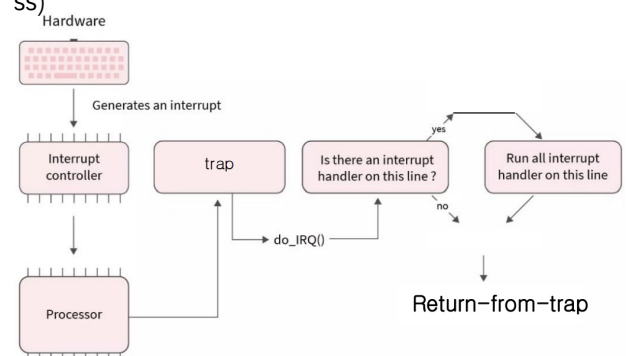
Return-from-trap instruction:

user program으로 되돌아옴.

특권 레벨 privilege level을 user level로 낮춤.

trap이 발생했을때 어떤 코드를 실행해야 하는가?

trap 발생 원인 : disk I/O의 완료, 키보드 interrupt, system call 등...



trap handler : 각 인터럽트 번호(trap number, 트랩 넘버)마다 실행할 코드. 실제 사건을 처리하는 코드.

- ◆ Interrupt number for **system call** : 0x64 @ xv6 0x30 @Pintos
0x80 @ linux

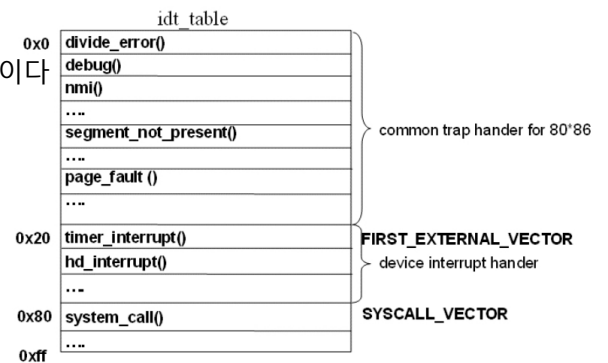
개념	뜻	무엇을 구분하나	예시	핵심 포인트
trap source	왜 커널로 들어왔는지 나타내는 사건 원인	커널 진입 원인	keyboard interrupt, disk I/O completion, system call	“왜 trap이 발생했는가?”를 설명하는 단계
trap number / interrupt number	그 사건 종류를 식별하는 번호	어떤 종류의 trap/interrupt인지	xv6 system call: 0x64, Pintos: 0x30, Linux 예시: 0x80	CPU/OS가 이 번호를 보고 어느 handler로 갈지 결정
trap handler	해당 번호에 대응해서 실행되는 커널 코드	들어온 사건을 실제로 처리하는 코드	키보드 처리 handler, 시스템 콜 진입 handler, 타이머 handler	“무슨 코드를 실행할 것인가?”에 대한 답
system call number	system call 안에서 구체적으로 어떤 요청인지 구분하는 번호	어떤 시스템 콜 서비스인지	read, write, fork, exec	system call로 커널에 들어온 뒤 한 번 더 세부 분기할 때 사용

Trap table

Trap handler들의 주소를 담고 있는 표

OS는 부팅할 때 trap table의 위치를 하드웨어(CPU)에게 trap table의 위치를 알려준다

- + trap table 위치를 알려주는 명령 자체도 privileged instruction이다
- + 이 명령은 user mode에서는 실행할 수 없다



문제 2 : 프로세스 간 전환

OS가 프로세스를 전환하기 위해서 어떻게 CPU의 control을 다시 빼앗아 올 것인가?

방법 1 : cooperative **협력적인** 접근 : 시스템 콜을 기다림.

방법 2 : Non-cooperative **비협력적인** 접근 : OS가 뺏어옴. (takes control)

협력적인 접근 Cooperative Approach

프로세스는 yield 같은 시스템 콜을 호출해서 주기적으로(periodically) CPU를 양보한다

OS가 다른 작업을 뭘 할지를 결정함.

애플리케이션이 뭔가 **불법적인(illegal)** 행동을 하면 **제어권이 OS로 넘어간다**

e.g. 0으로 나누기

e.g. 접근하면 안 되는 메모리에 접근

초기 Macintosh OS, 예전 Xerox Alto 시스템에서 이 방식을 사용했음.

-> 프로세스가 무한루프에 빠지면 머신을 재부팅해야한다. 방법이 없음..

비협력적인 접근 Non-Cooperative Approach

Time Interrupt 타이머 인터럽트

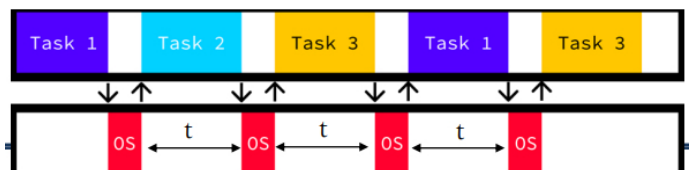
타이머는 미리 정해진 간격(몇 ms)마다 인터럽트를 발생시킨다.

interrupt가 raise되면..

현재 실행 중인 프로세스가 중단된다.

그 프로세스의 상태를 충분히 저장한다.

OS 안의 미리 설정된 interrupt handler가 실행된다.



-> 타이머 인터럽트는 OS가 CPU에서 다시 실행될 수 있는 능력을 준다(통제권 회복)

무한 루프 프로그램도 지금은 Control-C로 끊을 수 있는데, 그게 바로 OS가 CPU를 다시 통제할 수 있기 때문

스케줄러 : 현재 프로세스를 실행할지, 아니면 다른 프로세스로 switch할지 결정함.

switch를 하기로 결정하면, OS는 context switch를 함.

Context Switch

어셈블리 코드로 적힌 low-level 코드 조각.

context switch는 아주 자주 일어나고 매우 빠르게 실행되어야 하므로, 저수준 어셈블리 코드로 작성

1. 현재 프로세스의 레지스터 값들을 그 프로세스의 **kernel stack**에 저장한다

General purpose registers 범용 레지스터

PC(Program Counter)

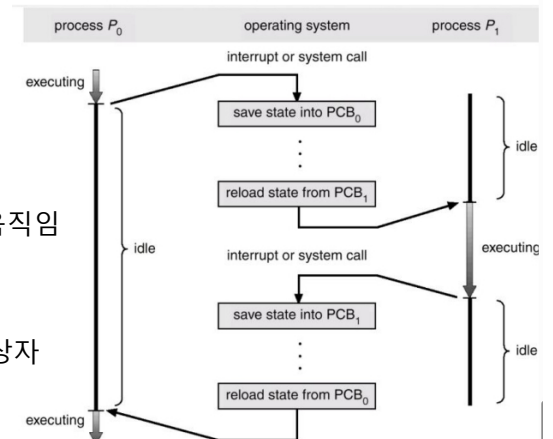
kernel stack pointer 현재 커널 스택의 어디까지 사용 중인지 가리키는 포인터

2. 다음 실행할 프로세스의 레지스터 값을 불러옴(restore).

다음 실행할 프로세스의 kernel stack으로부터 복구.

3. 다음 프로세스의 kernel stack으로 전환.

이제부터는 CPU는 다음 프로세스의 커널 context를 기준으로 움직임



PCB Process Control Block

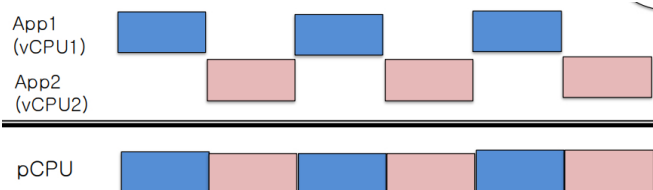
PCB : 프로세스를 다시 이어서 실행하기 위해 저장해 둔 요약 정보 상자

Process state : "running", "waiting", "ready"

Process ID(PID) : 모든 프로세스가 가지고 있는 unique한 ID

CPU Register : 프로세스의 CPU 레지스터 값은 그 프로세스의 상태를 복구하는 데 도움을 줌

Memory Information : base address나 총 할당된 메모리 양 같은 메모리 정보 (메모리 내용 전체를 넣는 것 X)



7주차 강의

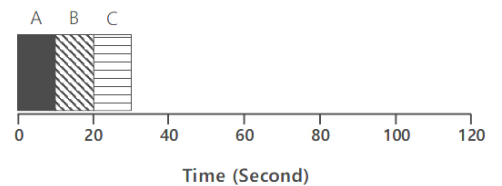
FIFO First Come First Out

매우 간단하고 구현하기도 쉬움.

First Come, First Served

- ♦ Arrived in order of A, B, C

- ♦ Each job runs for 10 seconds.



스케줄링 평가 기준(Metrics)

1. Turnaround time : 완료시점(completion) - 도착시점(arrival)
2. fairness(공정성) : 성능과 공정성은 상충될 수 있음. trade-off!

$$\text{Average turnaround time} = \frac{10 + 20 + 30}{3} = 20 \text{ sec}$$

지금까지의 예제는 이상적인 가정 위에 있다.

1. 모든 작업의 실행 시간이 같다.
2. 모든 작업이 동시에 도착한다.
3. 모든 작업은 CPU만 사용하고 I/O를 하지 않는다.
4. 각 작업의 실행 시간을 미리 알고 있다.

e.g. A=10 B=10

e.g. A, B 모두 0초에 도착

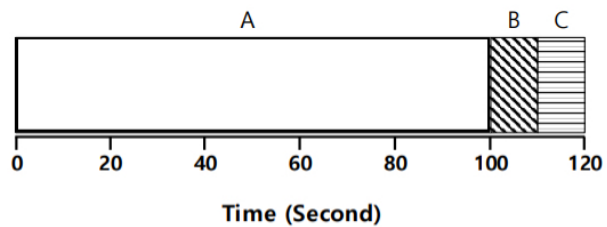
e.g. 모든 작업은 CPU만 쓰고 I/O를 하지 않음

e.g. 각 작업의 런타임을 이미 알고 있음

-> 이 수업에서 이 이상을 하나씩 완화할 것임

가정 1 완화 : job들의 실행시간이 달라졌다.

	Run time
A	100
B	10
C	10



$$\text{Average turnaround time} = \frac{100 + 110 + 120}{3} = 110 \text{ sec}$$

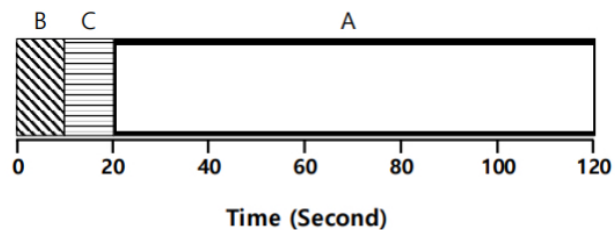
Convoy effect : Performance degradation when short job waits for long job

짧은 작업이 긴 작업을 기다리게 되면서 성능이 나빠지는 현상

Shortest Job First (SJF)

가장 짧은 작업을 먼저 실행.

	Run time len.
A	100
B	10
C	10

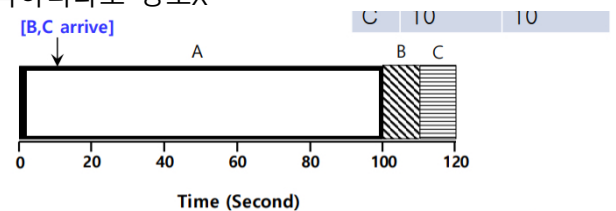


$$\text{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50 \text{ sec}$$

가정 2 완화 : Jobs can arrive at any time Job이 동시에 도착하지 않는다.

♦ SJF는 Non-preemptive scheduler여서 짧은 job이 나중에 도착하더라도 양보X

	Arrival time	Run time len.
A	0	100
B	10	10
C	10	10



$$\text{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec}$$

Shortest Time-to-Completion First (STCF)

SJF에 **preemption**한 버전.

aka Preemptive Shortest Job First (PSJF)

새 작업이 시스템에 들어오면: 현재 남아 있는 작업들과 새로 들어온 작업의 남은 시간을 비교

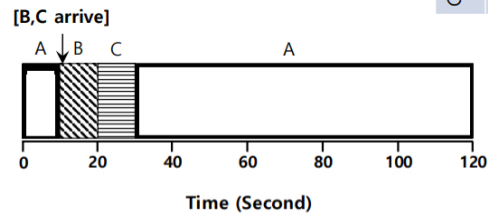
Schedule the job which has the **shortest time to complete**

끝나는 데까지 남은 시간이 가장 짧은 작업을 실행

Preemptive scheduler : HI-priority process can preempt LO-priority process

Priority가 높은 프로세스는 우선순위가 낮은 프로세스를 선점할 수 있다

	Arrival time	Run time len.
A	0	100
B	10	10
C	10	10



A는 0초에 도착, B, C는 10초에 도착
각각은 10초가 필요하다!

$$\text{Average turnaround time} = \frac{(120 - 0) + (120 - 10) + (120 - 10)}{3} = 50 \text{ sec}$$

새로운 scheduling metric : Response time

Response time : The time from when the job arrives to the **first time it is scheduled**.

작업이 도착한 순간부터, 처음으로 CPU에 배정될 때까지의 시간

$$T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$$

STCF와 그와 비슷한 방식들은 response time 측면에서는 그다지 좋지 않다

-> 어떻게 하면 Response time을 줄일 수 있을까?

Round Robin (RR) Scheduling

Time Slicing Schedule

: Run a job for a **time slice(time quantum)** and then **switch** to the next job in the **ready queue** until the jobs are finished.

하나의 작업을 **time slice(time quantum)** 동안 실행한 뒤, ready queue의 다음 작업으로 전환한다. 이것을 작업들이 끝날 때까지 반복한다.

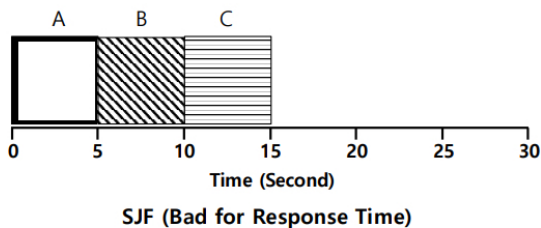
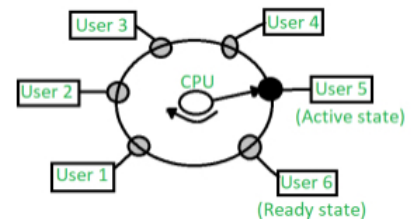
time slice의 길이는 timer interrupt 주기의 배수여야 한다.

e.g. 타이머 인터럽트가 1ms마다 오면 quantum을 1ms, 2ms, 5ms, 10ms 등으로 설정해야 함.

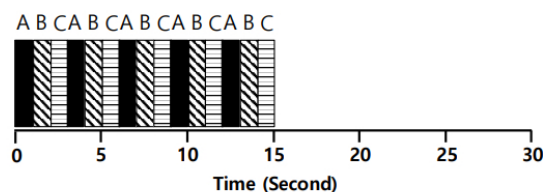
→RR is fair, but performs poorly on metrics such as turnaround time

RR은 공정하지만, turnaround time 같은 지표에서는 성능이 좋지 않다

- ▣ A, B and C arrive at the same time.
- ▣ They each wish to run for 5 seconds.



$$T_{\text{average response}} = \frac{0 + 5 + 10}{3} = 5 \text{ sec}$$



$$T_{\text{average response}} = \frac{0 + 1 + 2}{3} = 1 \text{ sec}$$

Time slice의 길이의 중요성

Time slice가 짧으면

응답 시간이 더 좋아진다.

context-switch 비용이 전체 성능을 지배하게 될 수 있다 오버헤드 발생.

Time slice가 길면

context-switch 비용을 분산시킬 수 있다

응답 시간은 더 나빠진다

-> time slice 길이를 결정하는 것은 시스템 설계자가 **trade-off**를 고려해서 설계해야 한다.

I/O를 포함

가정 3을 완화해보자. -> I/O를 자주 한다!

A와 B는 각각 50ms의 CPU 시간이 필요하다

A는 10ms 동안 실행한 뒤 I/O 요청을 한다 (I/O는 10ms가 걸린다)

B는 그냥 50ms 동안 CPU를 사용하고, I/O는 수행하지 않는다

컴퓨터에서의 I/O

옵션 1

CPU가 요청된 데이터를 디스크에서 메모리로 한 비트씩 직접 복사하는 방법
디스크 작업을 하는 동안 CPU는 다른 일을 할 수 없다

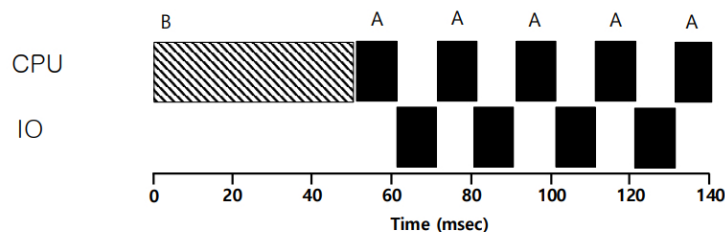
옵션 2

CPU가 I/O 장치에 I/O 요청만 보내는 방법

I/O 장치 컨트롤러가 독립적으로 디스크 작업을 수행한다 (디스크의 데이터를 메모리로 복사한다)

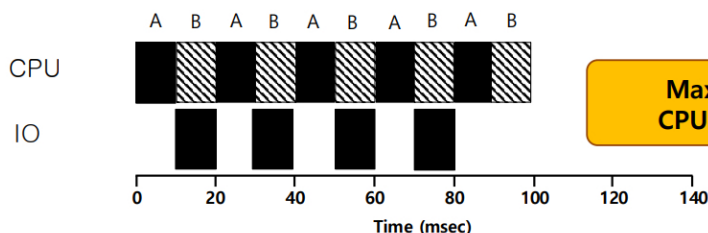
디스크 작업이 끝나면 I/O 장치 컨트롤러가 CPU에 인터럽트를 건다

-> 디스크 작업 동안 CPU는 다른 일을 할 수 있다



Poor Use of Resources

Approach 1:
B is higher priority than A



Overlap Allows Better Use of Resources

Approach 2:
A is higher priority than B

Maximize the
CPU utilization

오버랩이 허용됨!

>> 더 효율적

job이 I/O 요청을 시작하면

그 작업은 I/O 완료를 기다리며 **blocked** 상태가 된다

스케줄러는 CPU에 다른 작업을 배치한다.

I/O 가 완료되면

interrupt가 발생함.

OS는 그 프로세스를 blocked 상태에서 **ready** 상태로 다시 옮긴다

CPU 중심 작업과 I/O 중심 작업의 우선순위

CPU 중심 작업과 I/O 중심 작업의 분포

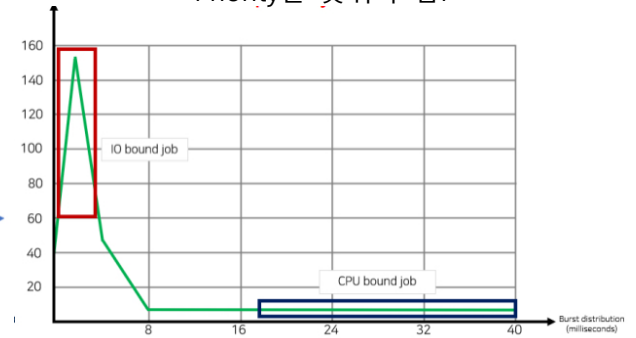
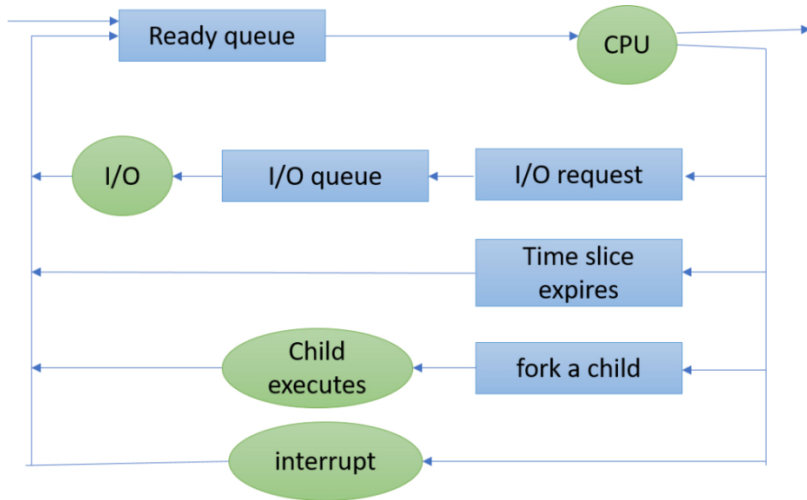
I/O intensive job : I/O를 기다리면서 CPU를 자주 양보하는 작업

CPU-intensive job : 오랜 시간 CPU를 계속 사용하는 작업

-> Priority를 높여야 함!

-> Priority를 낮춰야 함!

아하! I/O bound job이 더 많구나!->



Multi-Level Feedback Queue (MLFQ)

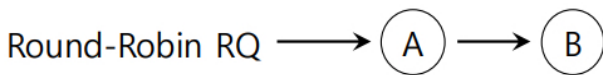
: 과거를 보고 미래를 예측하는 스케줄러

목표 : turnaround time을 최적화한다 → 더 짧은 작업을 먼저 실행한다
작업 길이를 미리 알지 못하는 상태에서 response time을 최소화한다

MLFQ는 여러 개의 서로 구별되는 큐를 가진다. 각 큐에는 서로 다른 Priority 레벨이 할당된다.
실행 준비가 된 job은 하나의 큐에 속해 있다. 더 높은 큐에 있는 작업이 실행 대상으로 선택된다.
같은 큐 안의 작업들 사이에서는 Round Robin을 사용한다.

Rule 1: If $Priority(A) > Priority(B)$, A runs (B doesn't).

Rule 2: If $Priority(A) = Priority(B)$, A & B run in RR.



MLFQ는 job의 관찰된 행동을 기반으로 Priority를 변경

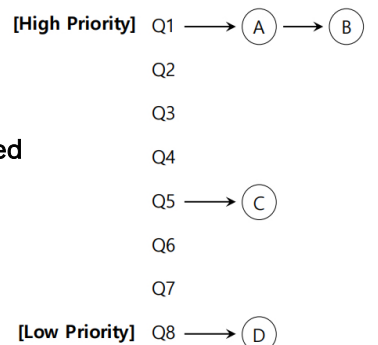
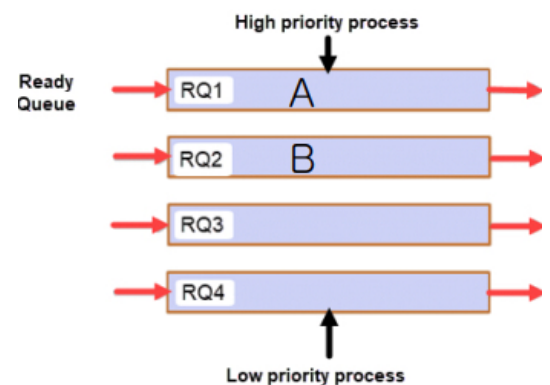
e.g. I/O를 기다리면서 CPU를 반복적으로 양보하는 작업 → Priority를 높게 유지한다

e.g. 오랜 시간 CPU를 집중적으로 사용하는 작업 → Priority를 낮춘다

Rule 3: When a job enters the system, it is placed at the highest priority

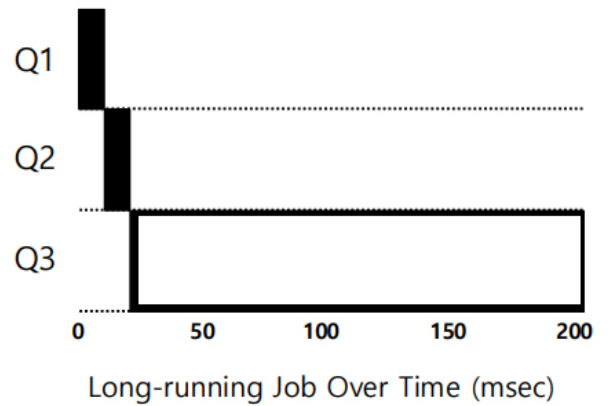
Rule 4a: If a job uses up an entire time slice while running, its priority is reduced

Rule 4b: If a job gives up the CPU before the time slice is up, it stays at the same priority level



e.g. time slice가 10ms인 상황.

time slice를 다 쓰니까 점점 priority가 낮아진다.



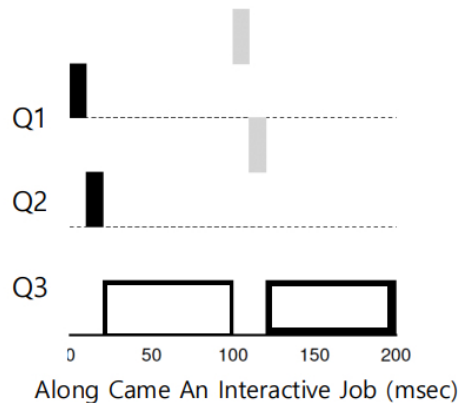
e.g. 짧은 작업이 들어온 상황

작업 A: 오래 실행되는 CPU 집약적 job

작업 B: 짧게 실행되는 인터랙티브 job (실행 시간 19ms)

A가 한동안 실행되고 있었고, 그 뒤 시간 T=100에 B가 도착한다.

-> job이 CPU를 양보하면 → 같은 priority를 유지한다

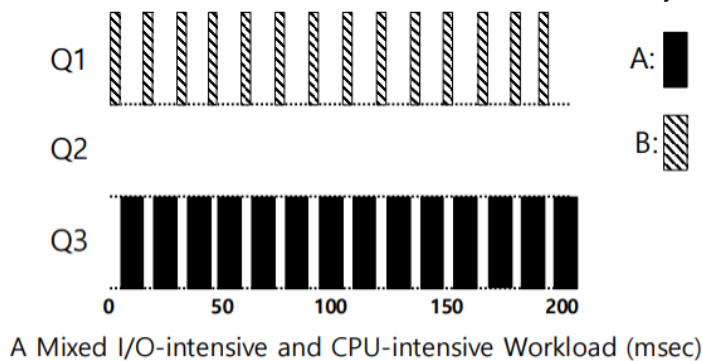


Rule 4b: If a job gives up the CPU → same priority

e.g. I/O가 있으면?

작업 A: 오래 실행되는 CPU 집약적 작업

작업 B: I/O를 수행하기 전 CPU를 1ms만 필요로 하는 interactive job



The MLFQ approach keeps an interactive job at the highest priority

>> MLFQ 방식은 interactive 작업을 가장 높은 priority에 유지한다

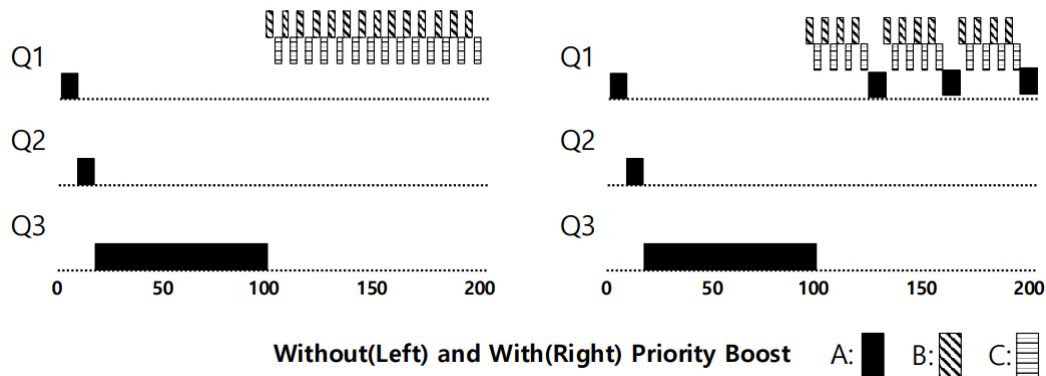
기본 MLFQ의 문제점

1. **Starvation** : 시스템에 interactive 작업이 “너무 많이” 있으면,
오래 실행되는 작업은 CPU 시간을 전혀 받지 못할 수 있다
2. **Game the scheduler** : time slice의 99%를 실행한 뒤 I/O 작업을 요청한다.
그 작업은 더 높은 비율의 CPU 시간을 얻게 된다
3. A program may change its behavior over time. 프로그램은 시간이 지나면서 동작이 바뀔 수 있다
CPU bound process → I/O bound process

Priority Boost

Rule 5: After some time period **S**, move **all the jobs** in the system to the **topmost queue**.

e.g. A long-running job(A) with two short-running interactive job(B, C)

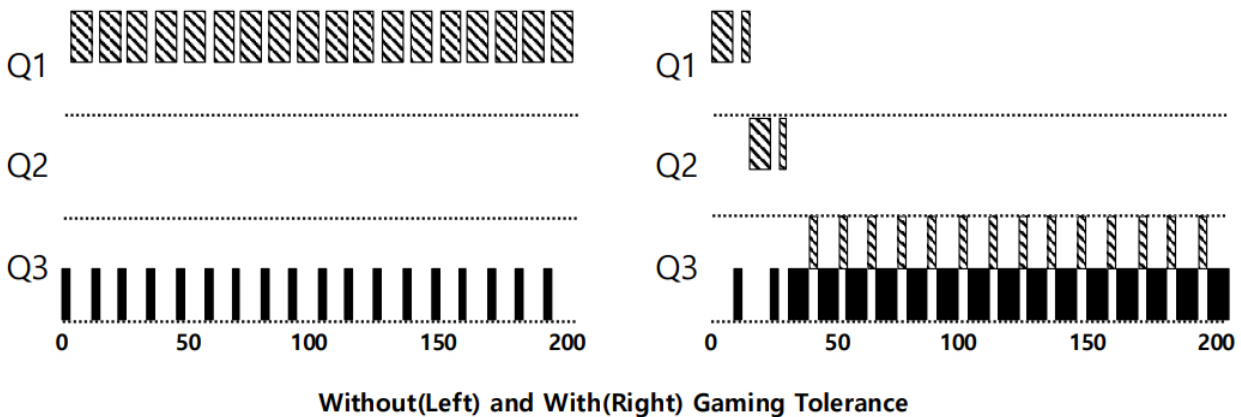


-> 문제 1(starvation), 문제 3(behavior change) 해결

Better Accounting

Rule 4 (Rewrite Rules 4a and 4b): Once a job uses up its **time allotment** at a given level (regardless of how many times it has given up the CPU), its priority is reduced(i.e., it moves down on queue).

한 큐 레벨에서 허용된 총 CPU 사용량(time allotment)을 다 쓰면, 중간에 몇 번 CPU를 양보했든 상관없이 아래 큐로 내린다.



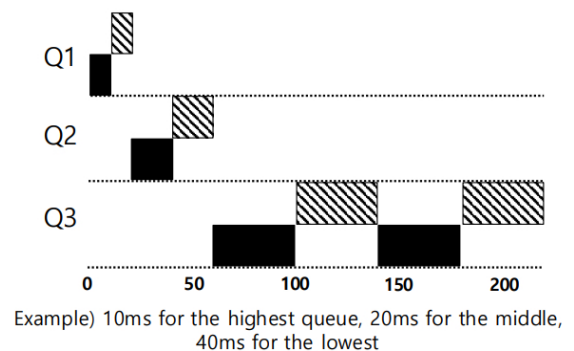
MLFQ 조정과 기타 문제들

우선순위가 낮을수록, 더 긴 time quantum!

->MLFQ에서 큐마다 time slice 길이를 똑같이 두지 않고, 우선순위에 따라 다르게 줄 수 있다

높은 우선순위 큐 → 짧은 time slice e.g. 10ms 또는 그보다 작게

낮은 우선순위 큐 → 긴 time slice e.g. 100 ms



Refine된 MLFQ rule

Rule 1: If **Priority(A) > Priority(B)**, **A runs** (B doesn't).

→ 규칙 1: Priority(A) > Priority(B) 이면, A가 실행되고 B는 실행되지 않는다.

Rule 2: If **Priority(A) = Priority(B)**, A & B run in **RR**.

→ 규칙 2: Priority(A) = Priority(B) 이면, A와 B는 RR(라운드 로빈)으로 실행된다.

Rule 3: When a **job enters** the system, it is placed at the **highest priority**.

→ 규칙 3: 작업이 시스템에 들어오면 가장 높은 우선순위에 배치된다.

Rule 4: Once a job **uses up its time allotment at a given level** (regardless of how many times it has given up the CPU), its priority is **reduced** (i.e., it moves down on queue).

→ 규칙 4: 작업이 어떤 레벨에서 자신의 time allotment를 모두 사용하면, CPU를 몇 번 양보했는지와 상관없이 우선순위가 낮아진다. (즉, 더 아래 큐로 내려간다.)

Rule 5: After some **time period S**, move all the jobs in the system to the **topmost queue**.

→ 규칙 5: 일정 시간 S가 지나면, 시스템의 모든 작업을 가장 위 큐로 올린다.

MLFQ의 장점

프로세스의 CPU 사용량에 대한 사전 지식을 필요로 하지 않는다.

-> 운영체제가 어떤 프로세스가 앞으로 CPU를 얼마나 오래 쓸지 미리 알 필요가 없다

2주차 실습

XV6 : small, educational, Unix-like OS

읽을 수 있고 이해가능하도록 디자인됨. 크지 않음.

OS에 대한 기본적인 것들, 어떻게 동작하는지를 배울 수 있음. 수정할 수도 있음.

이론을 구현해볼 수 있음. -> 학습용 OS이다

QEMU라는 가상 머신 위에서 **RISC-V** 머신을 에뮬레이션하여 구동됨.

XV6 주요 용어

OS : CPU/메모리/파일/장치를 관리함

Kernel : OS의 핵심(privileged core)

User Program : 앱/명령어 (e.g. ls, cat, sh)

Shell : command를 실행할 수 있는 프로그램 (sh)

System call : 유저 프로그램이 kernel 서비스를 호출하는 것 (e.g. 파일 I/O, 프로세스)

XV6 프로젝트 구조

kernel/ : OS 커널 코드(프로세스, 메모리, 파일 시스템, locks)

user/ : 유저 프로그램(xv6 안에서 돌릴 명령어)

Makefile : build rules 빌드 규칙 + **UPROGS list** 목록 (xv6 안에 포함시킬 사용자 프로그램 목록)

mkfs/ : fs.img(초기 파일 시스템 이미지) 생성

README : lab 명령어/노트

XV6 명령어

ls : 파일 목록 출력

cat [파일명] : 파일 내용 출력

echo [텍스트] : 문자열 출력

> : 출력을 파일로 리다이렉트(새로 생성/덮어쓰기)

mkdir [디렉터리명] : 디렉터리 생성

rm [파일명] : 파일 삭제

rm -r [디렉터리명] : 디렉터리 삭제 // -r recursive하게 삭제!

grep [텍스트] [파일명] : 문자열 검색

kill [프로세스 번호 PID] : 프로세스 종료 요청

sh : 명령을 읽고 실행하는 shell 프로그램

명령어 실행하기

'ls'를 type하면

sh(shell)이 'ls'라는 이름의 프로그램을 exec함. -> 프로그램이 프로세스가 되어 실행됨.

-> 프로그램이 kernel에 시스템 콜을 호출함. -> 결과 출력

sh도 유저 프로그램이다.

kernel은 프로세스/파일/메모리를 관리한다

새 프로그램 실행하기

1) **user/** 디렉터리에 새 C 파일 생성

- 예: **user/intro.c**

2) **Makefile**의 **UPROGS** 목록에 등록

- 예: **_introW**

3) **make qemu**로 다시 빌드

4) xv6 shell 안에서 **intro** 실행

실습 1

텍스트파일은 만들고 xv6에서 실행해보세요

- 1) make qemu 실행
- 2) **cat > me.txt** //파일 생성하기
- 3) 내용 입력
- 4) **Ctrl + D** //입력 종료
- 5) ls
- 6) cat me.txt

실습 2

kernel에 의해 출력되는 프로세스 정보를 관찰해 봅시다.

Ctrl + P :

- 1) 커널이 출력하는 프로세스 정보를 관찰
 - **Ctrl + P**를 눌러 process list 출력
 - **zombie** 프로그램 실행 //일부로 좀비 상태가 생기도록 만들어 놓음. zombie
 - 다시 **Ctrl + P**를 눌러 비교

zombie process : exited 했지만 parent가 wait() 하지 않아 아직 정리되지 않은 process

3주차 실습

Shell vs Console

console : 입력/출력이 보이는 창. 키보드와 화면이 연결되는 **입출력 공간**

shell : 그 안에서 명령을 해석하고 실행시키는 **프로그램**



xv6에서 프로그램이 작동하는 방식

1. 사용자가 console에 ls를 입력한다.
2. shell이 ls 프로그램을 찾는다.
3. ls가 user program(프로세스)으로 실행된다.
4. 실행 중 필요한 작업은 system call로 kernel에 요청한다.
5. 결과가 다시 console에 출력된다.

★ls, cat 같은 것은 커널 코드가 아니라 user program이다. 즉, 단순한 글자가 아니라 실제 실행 파일이다.

Host vs XV6

Host : macOS / WSL등

코드 수정(user/, Makefile 편집)

make qemu로 빌드 및 실행

xv6 : QEMU 안. \$가 떠 있음

실행파일(executables) 실행

vx6의 파일 시스템 이미지(fs.img)안에 있는 파일들이 보임

fs.img + UPROGS

fs.img : xv6가 부팅할 때 사용하는 파일 시스템 이미지. xv6가 들고 들어가는 “디스크” 같은 것

UPROGS : fs.img 안에 포함할 user program 목록

make qemu가 하는 일 :

user/*.c를 컴파일함.

UPROGS에 등록된 프로그램만 fs.img에 넣음

QEMU에서 xv6 부팅

CPU virtualization

: CPU 하나(또는 소수의 CPU)가 여러 프로그램을 동시에 돌리는 것처럼 보이게 만드는 것

실제 머신에는 몇개의 CPU 코어밖에 없지만, 한 번에 돌려야 할 프로그램은 많음.

자원이 한정되어 있을 때 어떻게 여러 개의 프로그램을 돌릴까?

Time Sharing : 짧은 시간동안 A를 실행

Scheuler : 누구를 다음에 돌릴지 결정

Context Switch : 현재 실행 중인 프로그램의 CPU 상태를 저장, 다음 프로그램의 CPU 상태 복원, 중간부터 자연스럽게 실행하는 방식

pingpong: fork()로 child process를 만들고 두 프로세스가 동작함

Ctrl + P: kernel이 process dump를 출력함

Host(mac OS/WSL)에서 사용하는 명령어

pwd : current directory (host only)

ls : list files/folders

cd [DIRECTORY_NAME] : change directory

make qemu: build + run xv6

make clean: clean build artifacts (only if needed)

xv6 shell (\$) 명령어

ls: list files

cat [FILE_NAME] : 파일 내용 출력

cat > [FILE_NAME] : 파일 생성/쓰기 (Ctrl + D로 종료)

mkdir [DIRECTORY_NAME] : 디렉토리 생성

rm [FILE_NAME] : 파일 삭제

ln [원본] [새로운 파일/바로그기] : hard link 생성

//이름(바로그기)을 하나 만들었다!

Ctrl + P: kernel process dump

실습1

: Host -> XV6

1. host에서 user/hello.c 생성
2. Makefile의 UPROGS에 \$U/_helloW 추가
3. make qemu로 다시 빌드
4. xv6 안에서 hello 실행

실습 2

Hard link : "파일을 하나 더 만드는 것"이 아니라 "같은 파일 데이터를 가리키는 이름을 하나 더 만드는 것"이다.

1. xv6 안에서 cat > note.txt 로 파일 생성
2. ln note.txt note2.txt 로 hard link 생성
3. rm note.txt 로 원래 이름 삭제
4. cat note2.txt 로 데이터가 남아 있는지 확인 -> 남아 있다! 접근 가능.

4주차 실습

Linux

UNIX 기반 오픈소스 OS. 1991년 리누스 토발즈에 의해 개발

서버 운영체제 : 리눅스는 웹 서버, 데이터베이스 서버, 파일 서버를 포함한 다양한 **서버 환경**에서 널리 사용

개발 환경 : 개발자들은 소프트웨어를 개발하고, 테스트하고, 배포하기 위해 리눅스를 사용

임베디드 시스템: 리눅스는 라우터, 스마트 TV, IoT 기기 같은 다양한 임베디드 장치의 운영체제로도 사용

데스크탑 : 우분투(Ubuntu), 리눅스 민트(Linux Mint) 같은 배포판은 일반 사용자에게 데스크톱 환경을 제공

CLI (Command Line Interface)

사용자가 키보드로 명령어를 입력해서 시스템과 상호작용하는 텍스트 기반 사용자 인터페이스

GUI와 비교했을 때, CLI는 마우스나 그래픽 요소 대신 텍스트와 명령어를 사용

e.g. bash, PowerShell, zsh

Command

작업을 수행하기 위해 입력하는 **기본 프로그램 또는 동작**

e.g. 파일 조작, 프로그램 실행, 설정 변경

Arguments(and options)

명령어 뒤에 추가로 붙는 **입력**으로, 무엇을 대상으로 동작할지 또는 어떻게 실행할지를 지정한다

```
cat cpu.c // arguments가 있다.
```

```
grep -n "cpu" cpu.c // 옵션이 붙는다!
```

Root, User, Group

Root : 리눅스의 **슈퍼유저** 계정, 시스템에 대한 모든 관리자 권한을 가짐

User : 일반적인 작업에 사용하는 표준 계정, 기본적으로 권한이 제한되어 있음

Group : 여러 사용자를 묶어 놓은 집합, 여러 사용자의 접근 권한을 한 번에 관리하기 위해 사용됨(공유 권한)

Permission

: 파일/디렉터리에 대한 접근을 제어하는 규칙(누가 무엇을 할 수 있는지)

읽기(r), 쓰기(w), 실행(x)

Permission Entities : 소유자(owner) / 그룹(group) / 기타 사용자(others)

Sudo

: 슈퍼 유저처럼 행동하기

특정 사용자에게 **일시적으로** 관리자의 권한을 부여하여 명령어를 실행할 수 있게 해 주는 명령어

/etc/sudoers는 누가 sudo를 사용할 수 있는지, 그리고 **어떤 명령어**를 실행할 수 있는지를 정의하는 설정 파일
아무나 sudo를 쓸 수 있는것은 아님.

명령어 앞에 **sudo**를 붙여 사용한다 (예: sudo [명령어])

밀번호를 입력하고 인증이 완료되면 명령어가 실행된다

Apt Advanced Package Tool

리눅스 시스템에서 소프트웨어 패키지 관리를 쉽고 효율적으로 해 주는 도구

사용자가 간단한 명령어로 패키지를 설치, 업데이트, 업그레이드, 제거할 수 있게 해 줌.

- **Package Installation:** `sudo apt install [package_name]`
- **Updating Package Lists:** `sudo apt update`
- **Package Upgrade:** `sudo apt upgrade`
- **Package Removal:** `sudo apt remove [package_name]`
- **Clean Removal:** `sudo apt autoremove`
- **Package Search:** `apt search [package_name]`

Command

- **ls:** List files
- **ll (ls -al):** Detailed list (permissions, owner, size..)
- **rm:** Remove files Trash X, irreversible!!
- **rm -rf [DIRECTORY_NAME]:** Remove directory (force)
- **whoami:** Current user
- **pwd:** Current directory
- **uname:** System info
- **~:** Home directory
- **man:** Command manual
- **mv:** File, Directory move/rename
- **cp:** File, Directory copy
- **clear (Ctrl + L):** Clear screen
- **grep:** Search text
- **tar:** Archive/compress
- **ps:** process status
- **export:** Set environment variable
- **echo:** Print text/variable

`cd [디렉터리이름]` : 지정한 디렉터리로 이동한다

`cd ..` : 한 단계 위의 부모 디렉터리로 이동한다

`cd ~` : 현재 사용자의 홈 디렉터리로 이동한다

`mkdir [디렉터리이름]` : 디렉터를 만드는 명령어

`rmdir [디렉터리이름]` : 빈 디렉터를 삭제하는 명령어

`rm -r [디렉터리이름]` : 비어 있지 않은 디렉터를 삭제

파일

vi, nano, vim

Basic text editor

To make file: `vi [FILENAME]`

`i:` to insert

Esc: to exit insert mode

`:wq:` to save and quit

`:q!` to quit without saving

cat

`cat` : 파일 내용을 출력하거나 이어 붙이는 명령어

touch

빈 파일을 만들거나 파일의 시간 정보를 갱신하는 명령어

CPU Virtualization

하나의 CPU가 여러 개인 것처럼 보이게 만드는 것

실제 컴퓨터는 CPU 코어 수가 몇 개 안 된다. 그런데도 많은 프로그램이 동시에 실행되는 것처럼 보인다

`gcc cpu.c -o cpu` //cpu.c를 컴파일

`./cpu "A"` //cpu 프로그램에 'A'라는 문자열을 매개변수로 넘겨서 사용

```
[(base) obyunghoon@obyeonghun-ui-MacBookPro projects % gcc cpu1.c -o cpu1
```

```
[(base) obyunghoon@obyeonghun-ui-MacBookPro projects % ./cpu1 A
```

```
A
```

```
A
```

```
A
```

```
^C
```



Foreground vs Background

Foregorund

터미널에서 직접 입력을 받고 출력을 화면에 표시한다
사용자와 직접 상호작용한다
사용자가 셸에서 명령을 실행하면, 셸이 그 명령의 실행이 끝날 때까지 기다리는 방식이다

Background

사용자 터미널과 독립적으로 실행된다
사용자가 다른 명령을 계속 실행할 수 있게 한다
사용자가 명령을 실행한 뒤 그 명령은 독립적으로 실행되고, 셸은 즉시 제어권을 사용자에게 돌려주는 방식

Lab 1 Permissions & sudo

`/etc/sudoers` (어떤 사용자가 sudo를 사용할 수 있는지)
`/etc/shadow` (사용자 계정의 비밀번호 정보와 관련된 매우 민감한 파일)
등등은 root 계정권한 없으면 보통 읽을 수 없음

-> sudo 안붙이면 **Permission denied**
-> sudo 붙이면 잘 나옴

Lab 2 CPU Virtualization

`gcc cpu.c -o cpu` //컴파일하기
`./cpu a & ./cpu b & ./cpu c & ./cpu d &` //& 백그라운드로 실행해라

`ps -ef` //현재 시스템에서 돌아가는 프로세스를 길게 보여 주는 명령
// -ef : 모든 프로세스, 전체 포맷으로

`kill [PID]` // e.g. kill 1234
`kill $(jobs -p)` // 프로세스 모두 종료

«» Bash

a
c
b
d
a
d
b
c
...

5주차 실습

Process

프로세스는 **실행 중인 프로그램**
디스크에 있는 프로그램은 **passive**
메모리에 있는 프로세스는 **active**
프로그램이 메모리에 올라와 CPU에서 실행 가능한 상태

프로세스가 가지고 있는 것은?

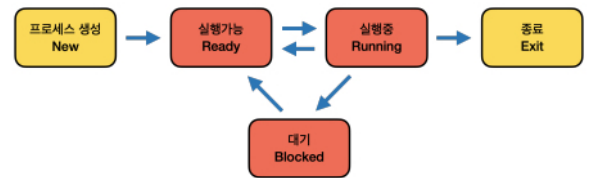
Memory : Instruction, Data
Register : PC, SP(stack pointer)

-> 프로세스는 코드와 실행 context를 모두 가지고 있다.

프로세스 Status

Running : 현재 CPU를 사용하는 상태
Ready : 실행할 준비는 되어있지만, 당장 실행중은 아님.
Blocked : I/O 같은 어떤 이벤트를 기다리고 있는 상태

Running → Ready : OS가 CPU에서 프로세스 제거
 Ready → Running : OS가 프로세스를 스케줄해서 CPU에 올림
 Running → Blocked : I/O 요청 등..
 Blocked → Ready : I/O 작업이 완료되었을 때.



프로세스 생성의 중요성

프로그램은 스스로 실행되지 않는다
 운영체제가 프로그램을 실행하려면 프로세스를 만들어야 한다. 프로세스 생성은 **실행 환경을 준비하는** 과정이다.
 → 여기에는 **메모리, 스택, 힙, 그리고 I/O 설정**이 포함된다.

프로세스 생성

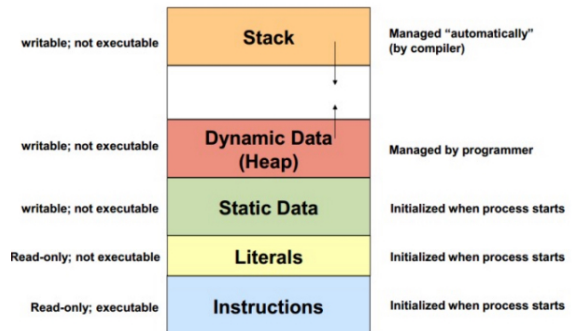
프로그램 loading : 디스크에 존재하는 프로그램을 OS가 메모리에 올림.
 -> 그 코드는 address space(주소 공간)의 일부가 됨.
 실행 도중 필요할 때마다 lazily하게 일어날 수 있다. 필요할 때, 필요한 부분만 가져오기

Run-time 스택 : OS가 run-time 스택을 할당함.

스택은 함수 매개변수, 지역변수, 리턴주소를 저장.
 프로그램은 초기화된 run-time 스택과 함께 시작됨

스택 vs 힙

스택 : 함수 관련 데이터 저장
 함수 호출, 리턴에 따라 커졌다가 줄어들.
힙 : 동적으로 할당된 데이터를 저장하는 데 사용됨
 메모리 할당과 해제(free)에 따라 커졌다가 줄어들



OS가 **힙**을 만들면, **input/output**을 초기화한다
 각각의 프로세스는 다음과 함께 시작됨 : Standard Input, Standard Output, Standard Error
 운영체제는 **엔트리 포인트(entry point)**에서 **실행**을 시작한다.

-> 프로세스에는 Code, (Static) data, Heap, Stack이 포함된다.
 운영체제는 CPU의 제어권을 새 프로세스에게 넘긴다.

파일 이름과 paths

사용자는 보통 파일 이름으로 파일에 접근한다.
 실제 대상 파일을 찾기 위해 주어진 경로를 해석(resolve)
path는 파일 내용 그 자체가 아님!
 파일에 접근하는 것은 **path resolution(경로 해석)**부터 시작됨.

Symbolic Link?

: 다른 경로를 가리키는 특별한 파일
 원본 파일 데이터를 복사해서 저장하지 않는다. -> 단지 대상 경로에 대한 참조(reference)일 뿐!

링크에 접근한다는 것은 대상 경로를 따라가는 것을 의미한다.

그러면 왜 Symbolic Link가 중요할까?

우리가 사용하는 파일 이름이 실제로 읽는 파일 자체가 아닐 수도 있다.
대상(target)이 바뀌면 같은 경로라도 다른 결과로 이어질 수 있다.
 Symbolic Link는 **path resolution**을 간접적으로 만들 수 있다.
 이 점은 리눅스 사용과 보안 측면 모두에서 중요

Symbolic Link의 작동 방식

대상 파일이 존재하면 link는 정상적으로 동작함.

이동되거나 삭제되면, 링크가 깨질 수 있다.

-> 링크 자체는 여전히 존재하지만, 그 링크가 가리키는 대상 경로는 더 이상 유효하지 않다.

최종 결과는 현재 대상 경로가 무엇이냐에 따라 달라진다.

Lab1 LINUX BASIC 2

```
gcc student_info.c -o student_info //파일 생성
./student_info "Sungmin" 12345678 "Sanbo" //이런식으로 매개변수를 전달해야 함.
ps -ef //10초동안 프로세스 살아있어서 확인 가능.
```

```
int main(int argc, char *argv[]) {
    printf("Name: %s\n", argv[1]);
    printf("Student ID: %s\n", argv[2]);
    printf("Department: %s\n", argv[3]);
    printf("PID: %d\n", getpid());
    sleep(10); //10초동안 프로세스가 살아있게 함.
    return 0;
}
```

Lab2 Symbolic Link

```
echo "SAFE DATA" > original.txt //original.txt 파일을 만들고 그 안에 "SAFE DATA" 를 쓴다.
ln -s original.txt link.txt //original.txt 를 가리키는 심볼릭 링크 link.txt 를 만든다.
```

```
cat link.txt //link.txt 를 읽어 본다.
```

```
mv original.txt moved.txt //original.txt 를 moved.txt 로 이름/경로 변경한다.
```

```
[(base) obyungmoon@obyonghun-ui-MacBookPro projects % echo "2025120431" > original.txt
(base) obyungmoon@obyonghun-ui-MacBookPro projects % ln -s original.txt link.txt

ln: link.txt: File exists
(base) obyungmoon@obyonghun-ui-MacBookPro projects % cat link.txt

2025120431
(base) obyungmoon@obyonghun-ui-MacBookPro projects % mv original.txt moved.txt

(base) obyungmoon@obyonghun-ui-MacBookPro projects % cat link.txt

cat: link.txt: No such file or directory
[(base) obyungmoon@obyonghun-ui-MacBookPro projects % cat moved.txt
2025120431
```

심볼릭 링크가 보이도록 ls -l 결과를 찍기

```
-rwxr-xr-x  1 obyungmoon  staff   50304  3 23 13:55 cpu
-rw-r--r--@ 1 obyungmoon  staff     501  3 23 12:58 cpu.c
-rwxr-xr-x  1 obyungmoon  staff   33752  3 23 15:10 cpu1
-rw-r--r--@ 1 obyungmoon  staff     600  3 23 15:34 cpu1.c
lrwxr-xr-x  1 obyungmoon  staff     12  3 30 13:50 link.txt -> original.txt
-rw-r--r--  1 obyungmoon  staff     11  3 30 14:00 moved.txt
-rwxr-xr-x  1 obyungmoon  staff   33520  3 30 13:25 student_info
-rw-r--r--@ 1 obyungmoon  staff     449  3 30 13:29 student_info.c
drwxr-xr-x 19 obyungmoon  staff     608  3 16 17:14 xv6-labs-2025
```

6주차 실습

프로세스 생성

프로그램은 디스크에 저장된 실행 가능한 파일

OS가 프로세스를 만들 때, load 하고, run-time 스택 만들고, heap 만들고, I/O 설정하고, 그다음에 main() 실행!



fork()

현재 프로세스를 복사해서 자식 프로세스를 만듦.

Return 값

parent: child PID

child: 0

failure: negative value

wait()

child가 끝날때까지 parent가 기다릴 때 사용

child가 존재하면 parent는 blocked됨

parent와 child의 실행 순서를 강제함.

- without wait() → parent/child output order may vary
- with wait() → child finishes first, then parent continues

exec()

현재 프로그램을 다른 프로그램으로 대체

새로운 프로세스를 만드는 것이 아님.

성공하면 원래 흐름으로 돌아오지 않음.

Shell 실행하면?

shell 자체가 그 명령어가 되는 것은 아니다.

shell은 자식 프로세스를 만들고, 그 자식 프로세스가 새로운 프로그램을 실행한다

File Descriptor

각 프로세스는 기본적으로 3개의 표준 파일 디스크립터를 가진다

파일 디스크립터 : 열린 파일이나 입출력 대상을 구분하기 위해 프로세스에게 붙여 주는 번호

0 : standard input (**stdin**) : keyboard / input

1 : standard output (**stdout**) : screen / output

2 : standard error (**stderr**) : error message

I/O Redirection

프로그램의 입력/출력 연결 대상을 기본값에서 다른 곳으로 바꾸는 것

자식 프로세스가 exec() 전에 stdout을 바꾸면, 프로그램의 출력은 화면 대신 파일에 써질 수 있다.

close(1)

//1번 디스크립터(출력)를 닫음.

open("out.txt", ...)

//out.txt를 열음. 출력은 앞으로 이곳으로 1번이 out.txt로 바뀜

exec("echo", args)

//프로그램 코드는 바꾸지만, 열린 파일 디스크립터 상태는 그대로 유지

//echo를 하면 out.txt에 출력이 됨

before: stdout(1) → terminal

after : stdout(1) → out.txt

리다이렉션 기호

- > : **stdout**을 파일로 보냄
- < : **stdin**을 파일에서 받음

Pipe

Pipe는 한 프로세스의 출력을 다른 프로세스의 입력으로 전달한다.

```
pipe(p)
echo hello world | wc           //echo hello world의 출력을 wc의 입력으로 받아서 처리

p[0] : read end
p[1] : write end
```

프로세스 A ----(쓰기 p[1])====파이프====(읽기 p[0])---- 프로세스 B

Lab1 Flow (Fork + Wait + Exec)

```
int main(int argc, char *argv[])
{
    int pid;
    printf("parent start: pid=%d\n", getpid());
    pid = fork();           // [빈칸 1] 자식 프로세스를 생성하시오.
    if(pid < 0){
        printf("fork failed\n");
        exit(1);
    }
    if(pid == 0){           // child
        printf("child: before exec, pid=%d\n", getpid());
        char *args[] = { "echo", "hello-from-exec", 0 };           // exec에 전달할 인자 배열      0은 인자 끝
        exec("echo", args);           // [빈칸 2] echo 프로그램을 실행하시오.
        printf("child: exec failed\n");           // exec가 성공하면 여기 도달하지 않음
        exit(1);
    } else {               // parent
        int wpid = wait(0);           // wpid에는 종료된 child PID가 들어감.  0은 별 의미 없음.
        printf("parent: child %d finished, wait returned %d\n", pid, wpid);
    }
    exit(0);
}
```

Lab2 I/O Redirection

```
int main(int argc, char *argv[])
{
    int pid = fork();           // 자식 프로세스를 생성하시오.

    if(pid < 0){
        printf("fork failed\n");
        exit(1);
    }
}
```

```

if(pid == 0){
    close(1);
    if(open("out.txt", O_CREATE | O_WRONLY) < 0){

        printf("open failed\n");
        exit(1);
    }
    char *args[] = { "echo", "this-goes-to-file", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
} else {
    wait(0);    // 부모 프로세스가 자식 종료를 기다리도록 하시오.
    printf("parent: child finished\n");
    printf("parent: now type 'cat out.txt' to check redirection\n");
}
exit(0);
}

```

// stdout, fd[1]을 받으시오.
//out.txt이 fd[1]을 차지하게 함
//O_CREATE : 파일이 없으면 생성
//O_WRONLY : 쓰기 전용으로 열기
// open의 반환값이 0보다 작으면(실패하면)
//failed 실행
//0은 인자가 끝났음을 표현.

7주차 실습

PCB

PCB : Process Control Block. OS가 각 프로세스를 관리하기 위해 사용하는 자료구조

OS는 동시에 많은 프로세스를 관리해야 한다.

PCB가 없다면,

프로세스들을 구별할 수 없음.

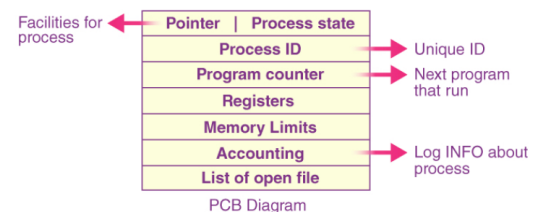
Status를 알 수 없음.

부모를 알 수 없음

프로세스를 복원할 수 없음.

pointer	process state
process number	
program counter	
registers	
memory limits	
list of open files	
.	
.	
.	

Process Control Block (PCB)



프로세스가 영원히 실행되지 않는 이유

OS는 때때로 현재 실행 중인 프로세스를 멈추고, CPU를 다른 프로세스에게 넘겨야 한다.

fairness, responsiveness, multitasking, system contrl 을 위해서..

Timer interrupt

타이머 인터럽트는 하드웨어가 주기적으로 만들어내는 신호이다.

현재 실행 중인 프로세스를 중단시키기 위함.

제어를 커널로 넘기기 위해

OS가 스케줄링 결정을 내릴 기회를 주기 위해

timer interrupt가 없다면..

하나의 CPU-bound 프로세스가 CPU를 독점할 수 있다.

시스템은 프로세스들 사이를 공정하게 전환할 수 없게 된다.

Context Switch

CPU가 한 프로세스에서 다른 프로세스로 넘어가는 과정

현재 프로세스 상태를 저장 -> 다른 프로세스를 선택한다. -> 다음 프로세스의 상태를 복원한다.

저장되는 정보는

PC : 다음에 어디부터 실행할지?

Register 값 : CPU안의 레지스터 값 저장

Stack Pointer : 함수 호출 상태, 지역변수 위치 등... 이걸로 어디로 돌아갈지를 알 수 있음.
그 밖의 실행 관련 상태

Timer Interrupt : OS가 control을 다시 얻어오는 것

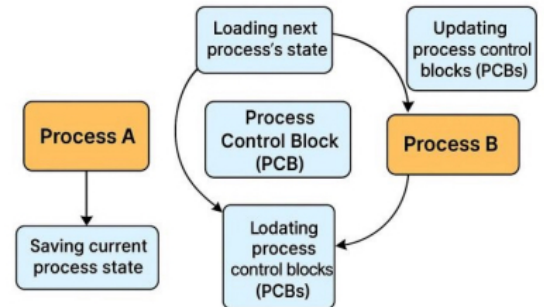
Context Switch : 현재 실행중인 프로세스를 바꾸는 것

FIFO

RR

Ready Queue -> 강의 참고

Context Switch



Lab 1 PCB

- Open kernel/proc.c and find the procdump() function.
- Copy and paste the provided modified procdump() code
- Run the system and press Ctrl-p to display process information
- Check whether the following fields are displayed correctly:
 - PID
 - PPID
 - STATE
 - SZ
 - NAME

\$	PID	PPID	STATE	SZ	NAME
1	0	0	sleep	20480	init
2	1	1	sleep	24576	sh

Lab 2 Timer vs Context Switch

```
#define READY 0
#define RUNNING 1
#define FINISHED 2
#define N 3

typedef struct {
    // {"P1", 3, READY}    프로세스 이름이 P1, 아직 3tick 남음, READY 상태
    char name[10];
    int remaining;
    int state;
} Proc;

void print_state(int state)
{
    if(state == READY)
        printf("READY");
    else if(state == RUNNING)
        printf("RUNNING");
    else if(state == FINISHED)
        printf("FINISHED");
    else
```

```

    printf("UNKNOWN");
}
int main(int argc, char *argv[])
{
    Proc p[N] = {                //프로세스 3개
        {"P1", 3, READY},
        {"P2", 3, READY},
        {"P3", 3, READY}
    };
    int finished = 0;            //끝난 프로세스 개수
    int cur = 0;                 //현재 실행할 프로세스의 인덱스
    int tick = 0;                //현재 시간
    printf("=== Timer Interrupt + Context Switch Simulation ===\n");
    while(finished < N){
        if(p[cur].state != FINISHED){
            p[cur].state = RUNNING;           //p[cur]가 RUNNING
            printf("tick %d: run %s (remaining=%d, state=", tick, p[cur].name, p[cur].remaining);
            print_state(p[cur].state);
            printf("\n");

            p[cur].remaining--;                // 1 tick 실행
            printf("tick %d: timer interrupt!\n", tick);    // tick 끝 -> timer interrupt 발생

            if(p[cur].remaining == 0){          //remaining이 0이면 끝 finished
                p[cur].state = FINISHED;
                finished++;
                printf("tick %d: %s finished (state=", tick, p[cur].name);
                print_state(p[cur].state);
                printf("\n");
            } else {
                p[cur].state = READY;           //remaining이 0이 아니면 Ready
                printf("tick %d: context switch -> save %s, switch next (state=", tick, p[cur].name);
                print_state(p[cur].state);
                printf("\n");
            }
        }
        tick++;
        cur = (cur + 1) % N;                  //cur을 순환시키기 위함!
    }
    printf("All processes finished.\n");
    exit(0);
}

```

Add one more initial process : 초기 프로세스를 하나 더 추가해라. 상수 3->로 늘림

Increase the execution time of any one process : 프로세스 하나의 시간을 3에서 5로 올림

Lab 3 Timer vs Context Switch

```

#define N 4
#define Q 1 // RR time quantum

```

```
typedef struct {
    char name[10];
    int arrival;      // 도착 시간
    int burst;        // 총 실행 시간
    int remaining;    // 남은 실행 시간
    int start_time;   // 처음 CPU를 받은 시간
    int finish_time;  // 종료 시간
    int first_run;    // 처음 실행했는지 여부 (-1이면 아직 안 함)
} Proc;
```

1. 상수와 구조체 정의

- 프로세스 개수 `N`
- RR의 time quantum `Q`
- 프로세스 정보를 담는 `Proc`

```
void copy_procs(Proc dest[], Proc src[], int n) { /* 원본 프로세스 배열 복사 */
```

```
    for (int i = 0; i < n; i++) {
        dest[i] = src[i];
    }
}
```

2. 보조 함수

- `copy_procs()`: 원본 배열 복사
- `print_metrics()`: 성능 지표 출력

```
void print_metrics(Proc p[], int n) { /* 결과 지표 출력 */
```

```
    printf("\nName\tArrival\tBurst\tFinish\tTurnaround\tWaiting\tResponse\n");
```

```
    for (int i = 0; i < n; i++) {
```

<code>int turnaround = p[i].finish_time - p[i].arrival;</code>	// turnaround	종료 - 도착
<code>int waiting = turnaround - p[i].burst;</code>	// waiting	턴어라운드 - burst(일한 시간)
<code>int response = p[i].start_time - p[i].arrival;</code>	// response	시작 - 도착

```
    printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i].name,
           p[i].arrival,
           p[i].burst,
           p[i].finish_time,
           turnaround,
           waiting,
           response);
}
```

```
} /* FIFO 시뮬레이션 */
```

```
void simulate_fifo(Proc original[], int n) {
```

```
    Proc p[N];
```

```
    copy_procs(p, original, n);
```

```
    int t = 0;
```

```
    printf("=== FIFO Scheduling ===\n");
```

```
    for (int i = 0; i < n; i++) {
```

<code>if (t < p[i].arrival) {</code>	//순서대로 실행
<code> t = p[i].arrival;</code>	//놓고있다면 점프

```
    }
```

```
    p[i].start_time = t;
```

```
    p[i].first_run = 1;
```

```
    printf("t=%d~%d : %s\n", t, t + p[i].burst, p[i].name);
```

```
    t += p[i].burst;
```

```
    p[i].remaining = 0;
```

```
    p[i].finish_time = t;
```

```
}
```

```
print_metrics(p, n);
```

```
}
```

3. FIFO 시뮬레이션

- `simulate_fifo()`

4. RR 시뮬레이션

- `simulate_rr()`

5. main

- 프로세스 4개를 준비하고
- FIFO, RR를 각각 실행

/* RR 시뮬레이션 */

void simulate_rr(Proc original[], int n) {

Proc p[N];

copy_procs(p, original, n);

int queue[100];

int front = 0, rear = 0;

int in_queue[N] = {0};

int finished = 0;

int t = 0;

printf("Wn=== Round Robin Scheduling (q=%d) ===Wn", Q);

while (finished < n) {

/* 현재 시각까지 도착한 프로세스 큐에 넣기 */

for (int i = 0; i < n; i++) {

if (p[i].arrival <= t && p[i].remaining > 0 && !in_queue[i]) {

이 조건은:

- 현재 시각까지 도착했고
- 아직 안 끝났고
- 현재 큐에 들어 있지 않은 프로세스면

큐에 넣을 후보라는 뜻이야.

int already_waiting = 0;

for (int k = front; k < rear; k++) {

if (queue[k] == i) {

already_waiting = 1;

break;

}

}

if (!already_waiting) {

queue[rear++] = i;

//큐에 추가

in_queue[i] = 1;

}

}

}

/* 준비 큐가 비면 시간만 증가 */

if (front == rear) {

//큐가 비면 시간만 증가

t++;

continue;

}

int idx = queue[front++];

//큐에서 하나 꺼내기

in_queue[idx] = 0;

//꺼냈으니까 해당 인덱스는 비어있다!

if (p[idx].first_run == -1) {

//한번도 CPU를 받아본 적 없으면

p[idx].start_time = t;

p[idx].first_run = 1;

- 남은 시간이 quantum보다 작으면 남은 시간만큼

- 아니면 quantum만큼

int run_time = (p[idx].remaining < Q) ? p[idx].remaining : Q;

예:

printf("t=%d~%d : %sWn", t, t + run_time, p[idx].name);

t += run_time;

p[idx].remaining -= run_time;

/* C

t=3~4 : P1

즉, 이 시간 구간 동안 P1이 실행됐다는 뜻.

/* 실행 후 새로 도착한 프로세스 큐에 추가 */

for (int i = 0; i < n; i++) {

if (p[i].arrival <= t && p[i].remaining > 0 && !in_queue[i]) {

int already_waiting = 0;

for (int k = front; k < rear; k++) {

//지금까지 아직 큐에 없는것 넣기

if (queue[k] == i) {

```

        already_waiting = 1;
        break;
    }
}
if (!already_waiting && i != idx) {
    queue[rear++] = i;
    in_queue[i] = 1;
}
}
}
if (p[idx].remaining > 0) {                //아직 안끝났으면 큐 뒤로
    queue[rear++] = idx;
    in_queue[idx] = 1;
} else {
    p[idx].finish_time = t;
    finished++;
}
}
print_metrics(p, n);
}int main(void) {
    Proc procs[N] = {
        {"P1", 0, 3, 3, -1, -1, -1},        //name, arrival, burst, start_time(처음 cpu 받음), finish, first_run
        {"P2", 1, 1, 1, -1, -1, -1},
        {"P3", 2, 4, 4, -1, -1, -1},
        {"P4", 3, 2, 2, -1, -1, -1}
    };
    simulate_fifo(procs, N);
    simulate_rr(procs, N);
    return 0;
}

```